

INFORME FINAL

Sistema de Gestión – Clínica Odontológica

Programación Orientada a Objetos – UADE 2026

1. Introducción

El presente informe documenta el desarrollo del sistema de gestión para una clínica odontológica, elaborado como trabajo práctico integrador de la materia Programación Orientada a Objetos (POO). El sistema permite administrar pacientes, odontólogos, secretarías y turnos a través de una interfaz gráfica completa desarrollada con Java Swing, con persistencia de datos en archivos de texto.

El proyecto fue desarrollado de forma incremental a lo largo de cuatro entregas:

- Entrega 1: Modelado de entidades y relaciones básicas.
- Entrega 2: Implementación de servicios con validaciones y lógica de negocio.
- Entrega 3: Manejo de excepciones personalizadas y mejoras en la capa de servicio.
- Entrega 4 (presente): Interfaz gráfica con Java Swing y persistencia en archivos.

2. Arquitectura del Sistema

El sistema implementa una arquitectura en capas siguiendo el patrón MVC (Modelo-Vista-Controlador):

Capa de Entidades (entity/): Contiene las clases de dominio del negocio. Todas implementan la interfaz `Serializable` como buena práctica de diseño en Java.

Capa de Repositorios (repository/): Gestiona el almacenamiento en memoria mediante estructuras `HashMap<Long, T>`. Cada repositorio implementa la interfaz genérica `IRepository<T>`, que define las operaciones CRUD básicas.

Capa de Servicios (service/): Contiene la lógica de negocio. Cada servicio implementa la interfaz genérica `IService<T>`. Las validaciones, reglas de negocio y manejo de excepciones residen en esta capa. La clase `Facturador` calcula los montos según el tipo de paciente.

Capa de Controladores (controller/): Actúa como intermediario entre la vista y los servicios. Recibe las solicitudes de la GUI, las delega al servicio correspondiente y retorna los resultados.

Capa de Vista (view/): Implementada completamente con Java Swing. MainFrame actúa como ventana raíz con un JTabbedPane de cuatro pestañas. Cada panel maneja su propio ciclo de vida (carga, alta, baja, modificación, búsqueda).

Capa de Persistencia (persistence/): La clase PersistenciaServicio guarda y carga los datos en archivos de texto plano con formato CSV (separador “;”) en la carpeta datos/: pacientes.txt, odontologos.txt, secretarias.txt y turnos.txt. Al cargar, resetea los contadores de ID estáticos al máximo encontrado para evitar colisiones de identificadores. Los turnos se cargan en último lugar, recibiendo los repositorios ya inicializados para resolver las referencias a Paciente, Odontólogo y Secretaria por ID; una vez construido cada Turno, se invoca agregarTurno() en cada entidad para reconstruir los historiales en memoria.

3. Decisiones de Diseño

3.1 Jerarquía de herencia en Odontólogo

Se definió Odontologo como clase abstracta con tres métodos abstractos: getEspecialidad(), getTarifaBase() y puedeAtender(String motivo). Esta decisión permite que cada especialidad (OdontologoGeneral, Ortodoncista, Endodoncista) defina su propio comportamiento sin duplicar código. La validación de compatibilidad entre el motivo de consulta y la especialidad del odontólogo se resuelve polimórficamente, sin necesidad de instanceof en la capa de servicio.

3.2 Interfaces genéricas IRepository<T> e IService<T>

Se utilizaron interfaces genéricas para formalizar el contrato de cada capa y facilitar la extensión del sistema. Esto permite agregar nuevas entidades sin modificar la arquitectura existente.

3.3 Manejo de excepciones personalizado

Se definió una jerarquía de excepciones que extiende RuntimeException: ClinicaException como base, y subclases específicas (DatoInvalidoException, PacienteNoEncontradoException, OdontologoNoEncontradoException, TurnoYaReservadoException). Todas son capturadas en la GUI y presentadas al usuario mediante JOptionPane.

3.4 Persistencia con Archivos de Texto (CSV)

Se eligió persistencia en archivos de texto plano (CSV con separador “;”) por su legibilidad, facilidad de depuración y compatibilidad con cualquier editor de texto. Cada entidad se guarda en su propio archivo dentro de la carpeta datos/. La carga sigue un orden estricto: primero Pacientes, Odontólogos y Secretarias; luego Turnos, cuyo método cargarTurnos() recibe los tres repositorios ya inicializados para resolver las referencias por ID. Al reconstruir cada Turno se invoca agregarTurno() en la entidad correspondiente, restaurando así los

historiales en memoria. Los contadores estáticos de ID se resetean al máximo cargado para garantizar la continuidad de los identificadores.

3.5 Sincronización de historiales

Cada alta de turno actualiza los historiales del paciente, odontólogo y secretaria involucrados (método `sincronizarAlta`). De igual forma, cada baja ejecuta `sincronizarBaja` para mantener la consistencia bidireccional de las relaciones.

3.6 Actualización de combos en TurnoPanel

Los JComboBox de paciente, odontólogo y secretaria se recargan cada vez que el usuario navega a la pestaña Turnos, mediante un `ChangeListener` en el `JTabbedPane`. Esto garantiza que los cambios realizados en otras pestañas se reflejen automáticamente sin necesidad de reiniciar la aplicación.

4. Patrones Aplicados

Patrón MVC (Model-View-Controller): Separación clara entre entidades (modelo), paneles Swing (vista) y controladores. Los paneles nunca acceden a los repositorios directamente; siempre pasan por el controlador.

Patrón Repository: Abstracción del acceso a datos detrás de la interfaz `IRepository<T>`, independizando la lógica de negocio de la implementación de almacenamiento.

Polimorfismo en Odontologo: El método `puedeAtender(String motivo)` se comporta distinto en cada subclase. `TurnoServiceImpl` llama a este método sin conocer el tipo concreto del odontólogo, delegando la decisión al objeto en tiempo de ejecución.

Comparable en Paciente: `Paciente` implementa `Comparable<Paciente>` para soportar ordenamiento natural por apellido y luego por nombre, utilizado en los JComboBox de turnos.

5. Diagrama UML

[VER DIAGRAMA UML ADJUNTO – generado como archivo separado]

El diagrama muestra la jerarquía completa de clases, interfaces, relaciones de herencia, implementación y dependencia entre las cinco capas del sistema.

6. Reflexión del Equipo

El desarrollo de este sistema permitió al equipo consolidar conceptos fundamentales de la programación orientada a objetos a través de un caso práctico realista.

La implementación incremental en cuatro entregas fue clave para comprender cómo los pilares de la POO se aplican en conjunto: la herencia redujo la duplicación de código en la jerarquía de odontólogos, el polimorfismo permitió validaciones flexibles sin estructuras condicionales rígidas, y las interfaces genéricas dotaron al sistema de extensibilidad real.

La integración de Java Swing en la entrega final representó el mayor desafío: gestionar el Event Dispatch Thread (EDT), coordinar el estado entre pestañas y manejar la persistencia en archivos de texto con resolución de referencias cruzadas y contadores estáticos fueron problemas que requirieron una comprensión profunda del lenguaje y del framework.

El proyecto en su conjunto demostró que una arquitectura bien organizada desde el inicio facilita considerablemente la incorporación de nuevas capas y funcionalidades, validando la importancia del diseño previo en el desarrollo de software.